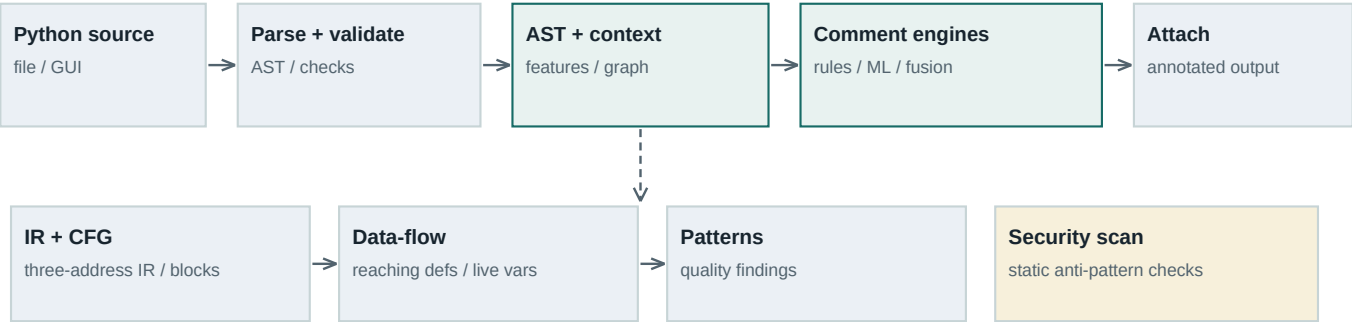


Existing Architecture

Implemented architecture evidenced in project documentation and source code.



Pipeline logs and PyQt6 workspaces display generation, analysis, and security results.

Current implementation

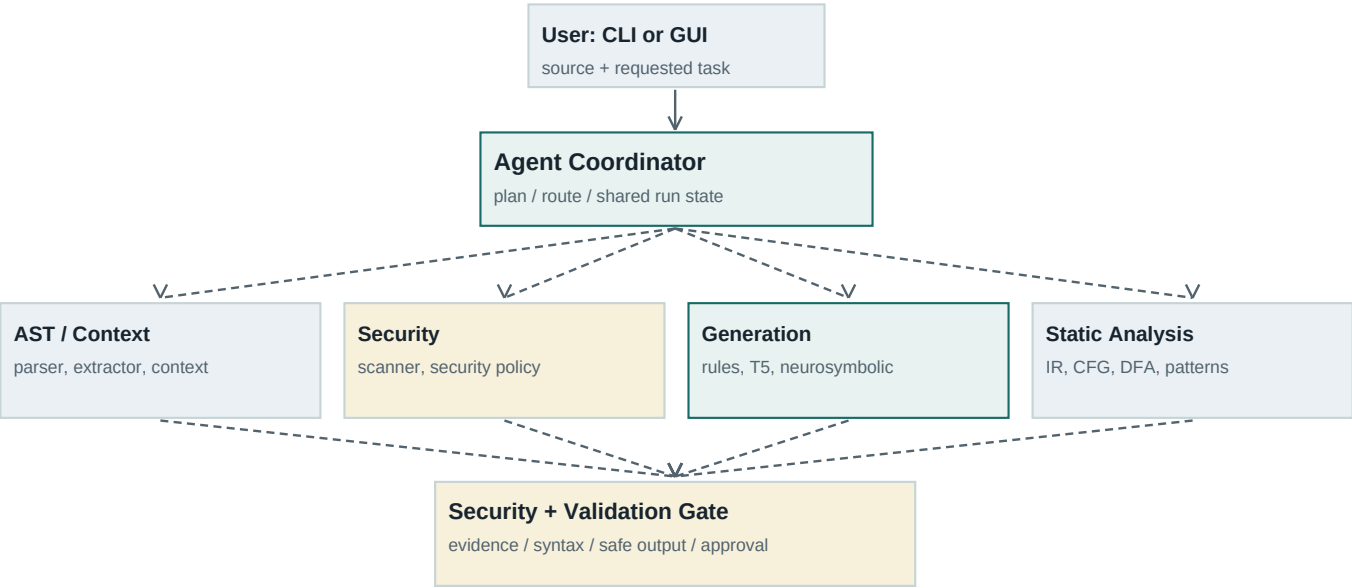
The application is a sequential Python source-analysis pipeline. The CLI and PyQt6 GUI pass source code to the parser and validator; AST features and context then feed the selected comment engine. Generated comments are attached to source. The pipeline also builds an intermediate representation, runs control-flow and data-flow analysis, detects code patterns, and produces a security report.

Layer	Implemented modules / responsibility
Input and front end	main.py, parser_module.py, validator.py: CLI orchestration, source parsing, AST checks. gui/: desktop interface and workspaces.
Program understanding	ast_extractor.py, ast_body_extractor.py, context_analyzer.py: AST features, function details, complexity, variables, and call graph.
Comment generation	comment_generator.py and ml/: rule-based and T5 AST+NLP generation. neurosymbolic/: confidence-gated ML, symbolic fallback, consistency checks.
Static analysis	ir/ and analysis/: intermediate representation, CFG, reaching-definitions/live-variable DFA, and pattern findings.
Security and reporting	security_analyzer.py: AST/context-based anti-pattern scan and remediation report. logger.py and GUI workspaces present results.

Security is already implemented and surfaced in CLI/GUI results. It is static, pattern-based analysis, not a complete security audit.

Proposed Architecture

Add coordinated, bounded agents across existing modules with explicit security controls.



Approved results return to comment attachment, reports, logs, and the GUI.

Proposed agentic operating model

A coordinator receives a user task, creates a bounded plan, and invokes specialist agents through allow-listed project tools. Agents reuse the current parser, analyzers, generation engines, and report builders. Results and source evidence are collected in shared run state, then pass security and validation gates before comments or files are changed. Decisions are logged for review.

Proposed agent	Reuse / responsibility	Control
Coordinator	Plan tasks, route work, track state, handle failures, and produce a traceable run summary.	Bounded plan; no unrestricted shell or network.
AST / context agent	Call parser, validator, feature extraction, and context analysis; return structured evidence.	Read-only source access.
Security agent	Run current scanner, prioritize findings, recommend fixes; later add dependency/config checks.	No automatic fix without validation and approval.
Generation agent	Select rule, ML, or neurosymbolic engine; draft comments from verified AST facts.	Sanitize output; never execute generated code.
Static analysis agent	Use IR, CFG, DFA, and pattern detection; summarize quality findings.	Structured tools with time/resource limits.
Validation / review gate	Check syntax, placement, consistency, security, and requested scope before export.	Approval for source changes; auditable logs.

Agent planning/orchestration is proposed, not currently implemented. Existing security, ML, and neurosymbolic modules are implemented capabilities.